

APPENDIX A

MC68010 LOOP MODE OPERATION

In the loop mode of the MC68010, a single instruction is executed repeatedly under control of the test condition, decrement, and branch (DBcc) instruction without any instruction fetch bus cycles. The execution of a single-instruction loop without fetching an instruction provides a highly efficient means of repeating an instruction because the only bus cycles required are those that read and write the operands.

The DBcc instruction uses three operands: a loop counter, a branch condition, and a branch displacement. When this instruction is executed in the loop mode, the value in the low-order word of the register specified as the loop counter is decremented by one and compared to minus one. If the result after decrementing the value is equal to minus one, the result is placed in the loop counter, and the next instruction in sequence is executed. Otherwise, the condition code register is checked against the specified branch condition. If the branch condition is true, the result is discarded, and the next instruction in sequence is executed. When the count is not equal to minus one and the branch condition is false, the branch displacement is added to the value in the program counter, and the instruction at the resulting address is executed.

Figure A-1 shows the source code of a program fragment containing a loop that executes in the loop mode in the MC68010. The program moves a block of data at address SOURCE to a block starting at address DEST. The number of words in the block is labeled LENGTH. If any word in the block at address SOURCE contains zero, the move operation stops, and the program performs whatever processing follows this program fragment.

	LEA	SOURCE, A0	Load A Pointer To Source Data
	LEA	DEST, A1	Load A Pointer To Destination
	MOVE.W	#LENGTH, D0	Load The Counter Register
LOOP	MOVE.W	(A0);pl, (A1)+	Loop To Move The Block Of Data
	DBEQ	D0, LOOP	Stop If Data Word Is Zero

Figure A-1. DBcc Loop Mode Program Example

A

The first load effective address (LEA) instruction loads the address labeled SOURCE into address register A0. The second instruction, also an LEA instruction, loads the address labeled DEST into address register A1. Next, a move data from source to destination (MOVE) instruction moves the number of words into data register D0, the loop counter. The last two instructions, a MOVE and a test equal, decrement, and branch (DBEQ), form the loop that moves the block of data. The bus activity required to execute these instructions consists of the following cycles:

1. Fetch the MOVE instruction.
2. Fetch the DBEQ instruction.
3. Read the operand at the address in A0.
4. Write the operand at the address in A1.
5. Fetch the displacement word of the DBEQ instruction.

Of these five bus cycles, only two move the data. However, the MC68010 has a two-word prefetch queue in addition to the one-word instruction decode register. The loop mode uses the prefetch queue and the instruction decode register to eliminate the instruction fetch cycles. The processor places the MOVE instruction in the instruction decode register and the two words of the DBEQ instruction in the prefetch queue. With no additional opcode fetches, the processor executes these two instructions as required to move the entire block or to move all nonzero words that precede a zero.

The MC68010 enters the loop mode automatically when the conditions for loop mode operation are met. Entering the loop mode is transparent to the programmer. The conditions are that the loop count and branch condition of the DBcc instruction must result in looping, the branch displacement must be minus four, and the branch must be to a one-word loop mode instruction preceding the DBcc instruction. The looped instruction and the first word of the DBcc instruction are each fetched twice when the loop is entered. When the processor fetches the looped instruction the second time and determines that the looped instruction is a loop mode instruction, the processor automatically enters the loop mode, and no more instruction fetches occur until the count is exhausted or the loop condition is true.

In addition to the normal termination conditions for the loop, several abnormal conditions cause the MC68010 to exit the loop mode. These abnormal conditions are as follows:

- Interrupts
- Trace Exceptions
- Reset Operations
- Bus Errors

Any pending interrupt is taken after each execution of the DBcc instruction, but not after each execution of the looped instruction. Taking an interrupt exception terminates the loop mode operation; loop mode operation can be restarted on return from the interrupt handler. While the T bit is set, a trace exception occurs at the end of both the looped instruction and the DBcc instruction, making loop mode unavailable while tracing is enabled. A reset operation aborts all processing, including loop mode processing. A bus error during loop mode operation is handled the same as during other processing; however, when the return from exception (RTE) instruction continues execution of the looped instruction, the three-word loop is not fetched again.

Table A-1 lists the loop mode instructions of the MC68010. Only one-word versions of these instructions can operate in the loop mode. One-word instructions use the three address register indirect modes: (An), (An)+, and -(An).

Table A-1. MC68010 Loop Mode Instructions

Opcodes	Applicable Addressing Modes
MOVE [BWL]	(Ay) to (Ax) (Ay) to (Ax)+ (Ay) to -(Ax) (Ay)+ to (Ax) (Ay)+ to -(Ax) -(Ay) to (Ax) -(Ay) to (Ax)+ -(Ay) to -(Ax) Ry to (Ax) Ry to (Ax)+
ADD [BWL] AND [BWL] CMP [BWL] OR [BWL] SUB [BWL]	(Ay) to Dx (Ay)+ to Dx -(Ay) to Dx
ADDA [WL] CMPA [WL] SUBA [WL]	(Ay) to Ax -(Ay) to Ax (Ay)+ to Ax
ADD [BWL] AND [BWL] EOR [BWL] OR [BWL] SUB [BWL]	Dx to (Ay) Dx to (Ay)+ Dx to -(Ay)
ABCD [B] ADDX [BWL] SBCD [B] SUBX [BWL]	-(Ay) to -(Ax)
CMP [BWL]	(Ay)+ to (Ax)+
CLR [BWL] NEG [BWL] NEGX [BWL] NOT [BWL] TST [BWL] NBCD [B]	(Ay) (Ay)+ -(Ay)
ASL [W] ASR [W] LSL [W] LSR [W] ROL [W] ROR [W] ROXL [W] ROXR	(Ay) by #1 (Ay)+ by #1 -(Ay) by #1

NOTE: [B, W, or L] indicate an operand size of byte, word, or long word.

